DCPL AI-Tester

Project Handover & Modernization Guide

Version: 1.1
Date: July 31, 2026
Prepared For: IT / Engineering Team, Dimakh Consultants
Prepared By: Advanced Agentic Coding Team

1. Executive Summary

The **DCPL AI-Tester** is a high-performance web auditing and crawler application developed for Dimakh Consultants. It provides automated checks across ten critical dimensions: SEO, Performance, Accessibility, Responsiveness, Forms, Navigation, Security, Content, Branding, and Footer metrics. The application utilizes **Playwright** for robust headless browser actions and audits, paired with a React/Next.js frontend and a FastAPI backend.

This guide covers the system architecture, code refactoring details, recent bug resolution history, and local development configurations to facilitate a smooth project handover.

2. System Architecture & Tech Stack

The system is divided into two distinct services communicating over HTTP/REST:

- **Frontend**: Next.js (version 16.2.7), React (version 19.2.4), Tailwind CSS, Recharts for dashboard data visualizations.
- **Backend API**: FastAPI (Python), SQLAlchemy (ORM), SQLite (database), Uvicorn (ASGI web server).
- **Auditing Engine**: Playwright Python API running headless Chrome instances to crawl websites, capture screenshots, and perform technical checks.
- **PDF Generation**: ReportLab template generators for offline Client and Developer PDF report compilation.

Directory Structure Overview:

```
c:/DCPL_AI-Tester/
  ■■■ backend/                  FastAPI Application Root
  ■   ■■■ app/
  ■   ■   ■■■ database.py        DB connections / sessions
  ■   ■   ■■■ models.py          SQLAlchemy Models
  ■   ■   ■■■ config.py          Backend parameters & CORS
  ■   ■   ■■■ engine/            Audit runner, crawler & report PDF logic
  ■   ■   ■■■ routers/           FastAPI routers (auth, jobs, audits, reports)
  ■   ■■■ run.py                 Backend entrypoint script
  ■   ■■■ app.db                 SQLite database file
  ■■■ frontend/                 Next.js Application Root
      ■■■ src/app/
      ■   ■■■ dashboard/         Tester dashboard page
      ■   ■■■ developer/         Read-only developer dashboard
      ■   ■■■ audits/[id]/      Audit detail, graphs & PDF downloads
      ■■■ next.config.mjs       Next.js dev configuration
      ■■■ .env.local            Frontend environment file
```

3. Summary of Recent Bug Fixes

Four critical system and architectural bugs were resolved in the codebase during recent sprints:

A. Audit Run All-N/A Scoring Bug

Issue: When auditing multi-page sites (e.g. 74 pages), if any subpage failed to audit due to network timeout or crawler hiccups, it emitted a TECH_PAGE_LOAD_FAILED finding. The scoring logic incorrectly checked for *any* page failure in the run and zeroed out/flagged as N/A the entire website score profile, rendering the audit results useless.
Resolution: Modified calculate_health_scores in backend/app/engine/runner.py to validate the failure of the **homepage seed URL only** (using normalized URL strings with trailing slashes stripped). Subpage timeout issues are logged as standalone issues without wiping out the entire report scores.

B. Crawl Map Visualization Overlap & Drift

Issue: The SVG-based Radial and Horizontal Crawl Maps were cluttered and overlapping due to two conflicting layout engines running in parallel (one in useEffect, one at render-time). In addition, an active physics relaxation loop was causing nodes to drift uncontrollably.
Resolution: Removed the stale render-time layout computation in frontend/src/app/audits/[id]/page.js. Expanded the Radial map bounds to a **1400x1000** viewport with customizable ring spacings, and the Horizontal map to dynamically scale heights vertically at 36px per node. Disabled the physics ticks (physicsTicks: 0) to ensure layouts are stable and deterministic.

C. Cross-Origin (CORS) & Network Access Blocks

Issue: Testing the application over local Wi-Fi router networks failed because API endpoints were bound to local loopbacks, backend CORS policies rejected requests, and Next.js blocked hot-module reload dev server access.
Resolution: Configured the frontend .env.local to target the host machine's loopback, updated the allowed CORS origin lists in backend/app/config.py, and declared appropriate development network hosts in next.config.mjs to permit local dev previews across network boundaries.

D. Comparison Trends Polling Loop Performance Fix

Issue: Moving to the 'Compare Runs' tab triggered an infinite loop of AJAX queries, firing dozens of API requests per second to the /api/jobs/trends endpoint due to state mutation in a React hook dependency array.
Resolution: Implemented a useRef flag (compareRunsFetchedRef) to track request state cleanly, and removed the state variable from the dependency array in page.js to enforce a single execution per render context.

4. Developer Operations & Environment Commands

To run the application locally in development mode, execute the following commands:

1. Start Backend Server:

```
cd c:/DCPL_AI-Tester/backend python run.py
```

The API server starts locally at **http://127.0.0.1:8080**.

2. Start Frontend Server:

```
cd c:/DCPL_AI-Tester/frontend npm run dev -- -p 5173
```

The frontend application builds and runs at **http://localhost:5173**.

5. Default Project Credentials

For testing, database tables have been seeded and verified with the following local logins:

Role	Login Email	Password
Tester (Read/Write)	aditya.gunjal@dimakhconsultants.com	password123
Developer (Read-Only)	developer@dimakhconsultants.com	password123

This concludes the project handover documentation.